What Are Microservices? — Quick Notes

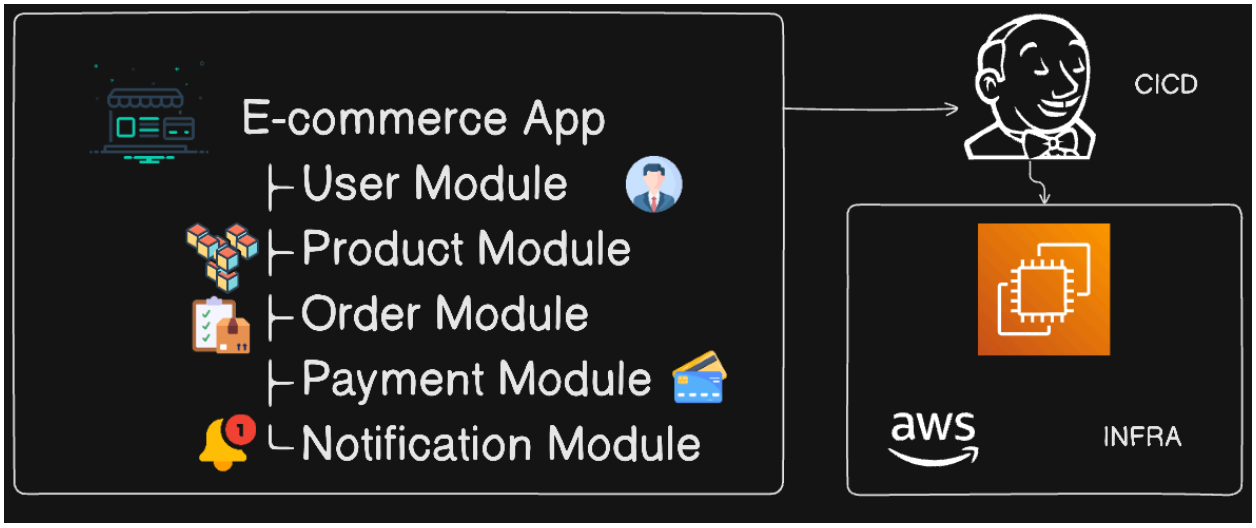
📌 Monolith Explained

A monolithic application is a **single, unified system** where all features are built and deployed together.

Example Modules:

- User Management
- Product Catalog
- Orders
- Payments
- Notifications

👉 Everything exists inside **one codebase and one deployment unit**



⚠️ Why Monolith Fails?

As the application grows, monolithic architecture starts facing several challenges:

❌ Hard to Scale

You must scale the **entire application**, even if only one module needs it.

❌ Hard to Deploy

A small change requires **redeploying the whole system**.

❌ Maintenance Issues

- Large codebase
- Slower development
- Difficult debugging

🚀 Microservices Explained

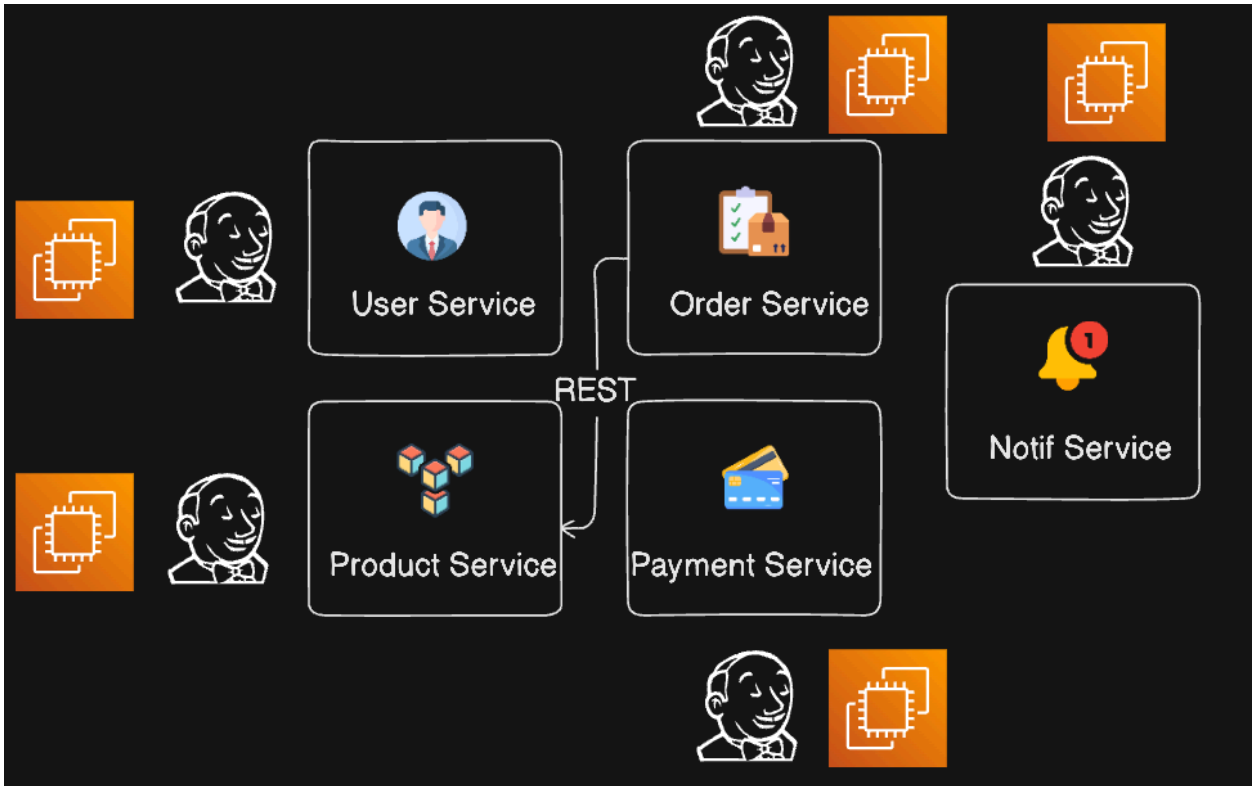
Microservices is an **architectural style** where an application is built as a collection of **small, independent services**.

Key Characteristics:

- Each service handles a **specific business function**
- Services are **independently deployable**
- Services can be **scaled independently**

Example:

- User Service
- Order Service
- Payment Service
- Inventory Service



🔗 Inter-Service Communication

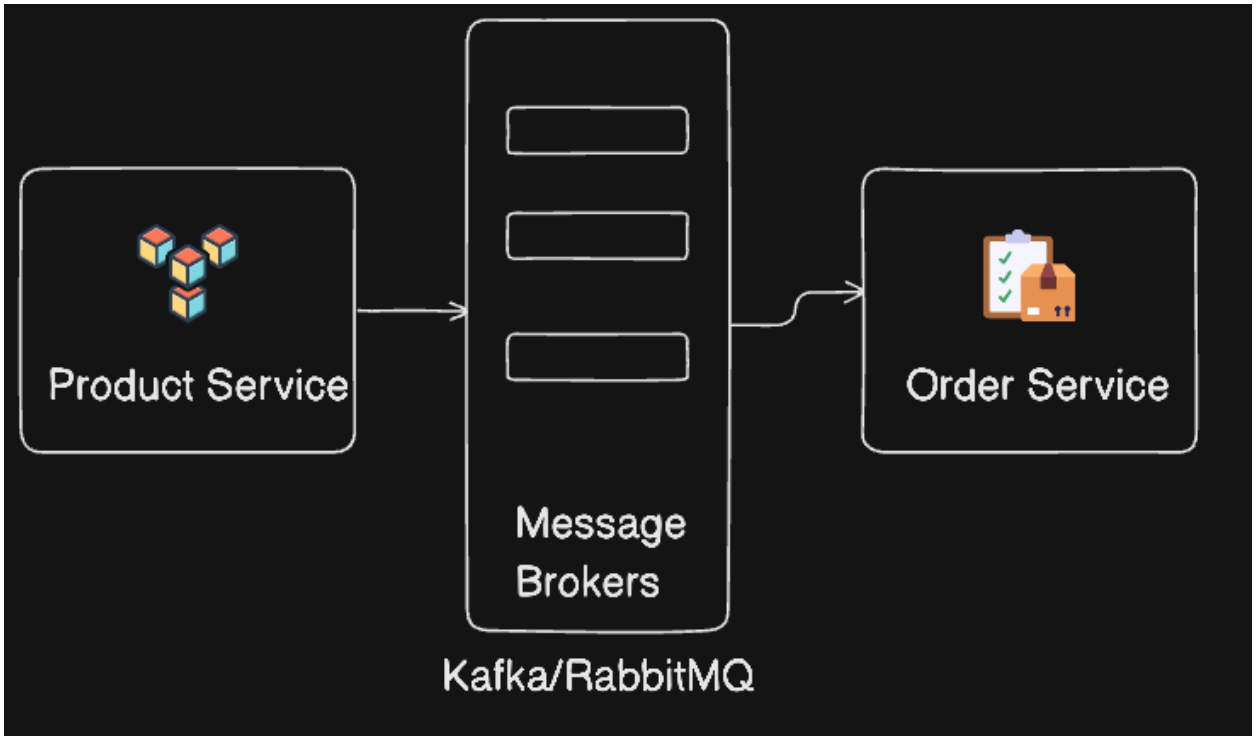
Microservices communicate with each other using:

1. Synchronous Communication

- Request → Response
- Example: REST APIs

2. Asynchronous Communication

- Event-based communication
- Example: Messaging systems (Kafka, RabbitMQ)



✅ When to Use Microservices

Use microservices when your system requires:

1. Large & Complex Applications

Multiple independent business domains.

---

2. Multiple Teams Working in Parallel

Example:

- Team A → User Service
- Team B → Order Service
- Team C → Payment Service

---

3. Independent Scaling

Scale only the services that need high traffic.

---

4. High Availability Systems

Failure in one service should not bring down the entire system.

---

5. Different Technology Requirements

Different services can use different tech stacks (Java, Node.js, Python, etc.)

❌ When to Avoid Microservices

Avoid microservices in the following scenarios:

❌ Small Applications

Simple systems don't need distributed complexity.

❌ Small Teams

Managing multiple services becomes difficult.

❌ No DevOps / Infrastructure Readiness

Microservices require:

- CI/CD pipelines
- Monitoring

- Logging
- Containerization

🎯 Key Takeaway

Microservices are powerful, but they are not always the right choice.
Start simple, and adopt microservices only when your system truly needs it.